Fleet monitoring architecture — proposal for Aaron approval

Status: Proposal only — no implementation until approved

Version: 1.1 (2026-08-14) — adds lane-safe repo path, live-status gating, edge-vs-origin probes, monitor identity/politeness, dead-man switch, publish-aware muting, cert/redirect checks, CrUX field CWV

Audience: Aaron (owner / security / infra) · Connect Group engineering

Scope: Continuous health, SEO crawl reliability, and alerting for all live Astro fleet sites

Registry source of truth: `scripts/fleet-sites.json` · `docs/FLEET-PORT-REGISTRY.md`

Related docs	Role
DEVELOPER-HANDOFF.md	Presentation publish lane
ASTRO-FLEET-AUTONOMOUS-RELEASE.md	Publish canary + public verify
10-GO-LIVE-VERIFY-CHECKLIST.md	Manual go-live matrix
SEO-AI-CRAWLER-AUDIT-REMEDIATION-2026-08-06.md	SEO handoff — monitoring still "future"
AARON-RULES.md	Protected paths + CWV debt notes

1. Why this exists

Deploy-time gates already exist (CI `fleet-validate`, local smoke, publisher canary + public verify). They are **change-gated** and do **not** continuously watch live HTTPS.

Without scheduled synthetic checks + access-log evidence:

- Intermittent **5xx** on a single URL can happen while the rest of the site looks fine
- Googlebot / crawler failure rates cannot be proven from WordPress or browser testing alone
- SEO / speed complaints become post-mortems instead of alerts

This proposal asks approval to build a **deliberate observability capability**, with clear ownership of data sources that sit outside the presentation lane (Cloudflare, Caddy logs, alert routing).

2. Goals and non-goals

Goals

1. Detect public HTTPS failures (2xx/3xx/4xx/5xx) across **all live fleet domains**
2. Detect latency regressions on critical routes
3. Continuously verify **pages + posts** discovered from sitemaps (not only homepages)
4. Separate **synthetic visitor checks** from **real crawler evidence** (verified Googlebot)
5. Distinguish **edge failures from origin failures** in every alert (Cloudflare vs Caddy/Node)
6. Alert humans with evidence (status, URL, site, commit header, timestamp) — not blame after the fact
7. The monitor must **never become its own incident**: identified traffic, rate-limited crawls, excluded from analytics, and self-monitored (a dead monitor must not look like a healthy fleet)
8. Fit the existing fleet model: registry-driven, evidence files, no secret sprawl

Non-goals (v1)

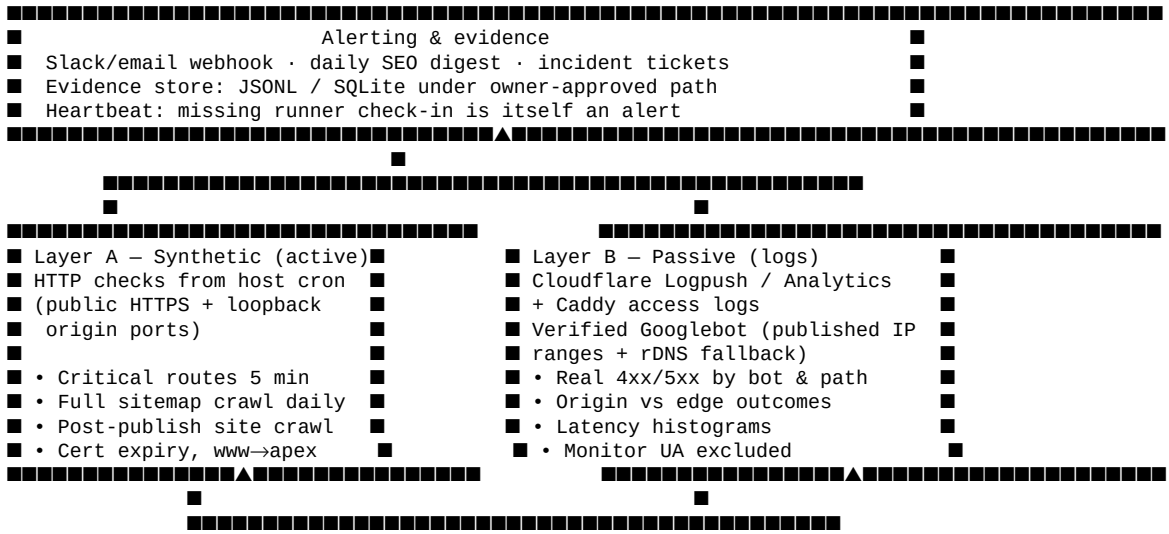
- Replacing Aaron's publisher canary
- Changing Caddy / systemd / Cloudflare policy without owner work
- Full browser Lighthouse on every post URL every minute (field CWV comes free from CrUX instead — § 7.3)
- Auto-submitting live contact forms on a schedule (creates junk Pipedrive leads)
- Training / scraping AI bots policy (business + Cloudflare decision)

3. Current state (honest)

Capability	Today	Gap
Registry of 42 domains	`scripts/fleet-sites.json`	Ready to drive monitors
Local smoke	`scripts/fleet-local.mjs smoke`	Status only; laptop/local; not continuous public HTTPS
CI probe	`scripts/ci/fleet-validate-site.mjs`	Changed sites only; `/`, robots, random 404
Publish canary	`astro-fleet-release publish`	Excellent at cutover; not 24/7
Per-site QA / sitemap scripts	Uneven across sites	Not one scheduled fleet job
Middleware `http_access` (status + ms)	Most sites	Process stdout only; no aggregation/alert
Continuous 5xx / Googlebot monitoring	**None in repo**	Needs logs + synthetic + alerts
Alerting (Slack/email/pager)	**None**	Needs owner-approved destination
Field Core Web Vitals visibility	Ad-hoc PSI runs (CHR LCP ~17.7s finding was manual)	No daily trend per origin

4. Recommended architecture (best approach)

Split into **three layers**. Layers 1–2 can start in engineering with low risk. Layer 3 needs Aaron's infra access.



```
scripts/fleet-sites.json
(domain, id, port, unit, status)
```

Why this is the best approach

Option	Verdict
Only extend publish canary	Necessary but insufficient — misses overnight / intermittent failures
Only Cloudflare dashboards	Good for bots/CDN; weak for "every sitemap URL still 200" and form JS regressions
Only per-site Playwright clones	Too expensive to run fleet-wide every few minutes
Registry-driven synthetic + log ingestion + alerts	Covers visitor truth + crawler truth; reuses existing registry and checklists

5. Layer A — Synthetic monitoring (active)

5.1 Site eligibility (registry `status` gating)

The registry carries a `status` per site (`scaffolded`, live, etc.). Monitoring **only enrolls sites whose status marks them live** (or an explicit `monitor: true` flag if we prefer opt-in). This prevents a day-one alert storm from scaffolded or not-yet-cut-over domains — the fastest way to get alerts ignored is to open with 40 false pages.

- **Live sites** → full T0/T1/T2 checks
- **Parked / redirect-only domains** → redirect assertion only (correct 301 target, valid cert)
- **Scaffolded / pre-launch** → excluded; enrolled automatically when status flips

5.2 URL discovery and check tiers

For each eligible site in `fleet-sites.json`:

Tier	Routes	Cadence
T0 Critical	`/`, `/robots.txt`, sitemap index, random guaranteed-404, `/api/health/` (if present), 3–5 conversion pages (contact / quote), **www→apex redirect**, **TLS cert days-remaining**	Every **5 minutes** (cert: daily)
T0b Origin probe	Same critical routes against `127.0.0.1:<port>` from the registry (host-cron placement only)	Every **5 minutes**
T1 Sitemap corpus	Recurse sitemap index → page + post locs	**Daily** (and after publish for changed sites)
T2 Browser journeys	Homepage + contact form render (no spam submit), key nav	Daily or post-publish only

Why T0b matters: all public checks go through Cloudflare, so a cached 200 at the edge can mask a dead origin for hours. The loopback probe (same mechanism as `fleet-local.mjs smoke`, same ports already in the registry) means every alert says **"origin down"** or **"edge problem"** — no guessing, and no waiting for cache expiry to discover a dead Node unit.

5.3 What each check records

- Site id / domain
- Request URL + final URL after redirects

- HTTP status · redirect count
- TTFB + total duration (ms)
- Response bytes + **content assertion result** (see 5.4)
- `cf-cache-status`, `x-astro-fleet-commit` (when present)
- Pass/fail vs thresholds
- Check type: `critical` | `origin` | `sitemap` | `browser` | `cert` | `redirect`

5.4 Content assertions (blank-200 defense)

A site serving an empty page or an error template with HTTP 200 passes every status-only check. Each T0 page check therefore also asserts:

1. **Minimum body size** (per-site threshold, calibrated in the baseline week)
2. **HTML marker** — e.g. `<title>` contains the site's brand string (one line per site in `thresholds.json`)

Cheap to run, catches a whole failure class that status codes cannot.

5.5 Suggested thresholds (tunable)

Signal	Suggested start
Homepage / robots / sitemap / health	Must be **200** (or documented intentional redirect once)
www hostname	Must **301** → **apex** (matches paired apex/www Caddy loaders)
TLS certificate	Warn < **14 days** remaining; fail < **7**
Critical TTFB	Warn > **1.5s**, fail > **4s** (adjust per site after baseline week)
Body size / marker	Fail if below floor or marker missing (per-site)
Sitemap URL	Fail on **5xx**; warn on unexpected **404** for previously-200 URL
Sitemap URL-count drop	Warn if drop > **5%** vs yesterday (first run seeds baseline, no alert)
Consecutive critical fails	Alert on **2 consecutive** fails (**3** during a publish window — see 5.8)

5.6 Monitor identity & crawl politeness

A scripted client hitting 42 domains every 5 minutes looks exactly like the bots Cloudflare is configured to challenge. To avoid false 403s — and to avoid polluting the very Layer B analytics we want to read:

1. **Fixed User-Agent:** `astro-fleet-monitor/1.0 (+internal)` on every request
2. **Owner ask:** a Cloudflare WAF skip/allow rule for the runner's egress IP + UA (one rule, owner-installed)
3. **Layer B exclusion:** all log metrics filter out the monitor UA so synthetic traffic never inflates real-visitor or bot stats
4. **Politeness:** sitemap crawls run with a concurrency cap (e.g. 4 in-flight per site), per-site rate limit, and jitter — a daily recursion across 42 sitemaps is thousands of URLs and must not become its own load spike or tripwire

5.7 Runner self-monitoring (dead-man switch)

If the cron dies, silence looks identical to health. Every runner cycle writes a heartbeat (evidence file timestamp and/or ping to a check-in endpoint). A missing heartbeat beyond 2× the expected interval is itself a **P2 alert** from whichever side is still alive (or, minimally, the daily digest states "monitor uptime" so a silent gap is visible within 24h).

5.8 Publish-aware muting

`astro-fleet-release publish` cutover can produce transient blips that a 5-minute check will catch and page on. After a publish for a site, that site's alerts are suppressed (or the consecutive-fail requirement raised from 2 → 3) for a short window (e.g. 15 minutes). Detection: the `x-astro-fleet-commit` header change, or a post-publish touch file. A false P1 on every release trains people to ignore pages — this rule protects alert credibility.

5.9 Forms (important)

Do **not** auto-POST production `/api/contact/` every N minutes.

Instead:

1. Assert form client assets load (e.g. `astro-mirror-fixes.js` / CF7 bridge) and contain correct `/api/contact/` trailing slash where required
2. Assert `/api/health/` 200
3. Real submit only: post-publish smoke with a **clearly tagged monitoring lead**, or an owner-approved dry-run route

5.10 Where the runner lives

Placement	Pros	Cons	Ask
A. Host cron under `webdev` (read-only public HTTPS + loopback probes)	Same network as fleet; enables edge-vs-origin discrimination (T0b); simple	Needs host crontab approval	**Preferred**
B. External runner (GitHub scheduled workflow / small VPS)	Outside host blast radius	Needs secrets + egress; no loopback probes; Actions not ideal for true uptime	Fallback for Layer A public checks only
C. Laptop-only	Zero infra	Not reliable 24/7	Dev / baseline only

Recommendation: Approve **A**. Outbound HTTPS to public sites + read-only loopback GETs, no Caddy edits, no writes outside the approved evidence path. The loopback probe capability is the strongest argument for A over B: it is what turns "site is down" into "origin unit X is down, edge still serving stale."

5.11 Proposed repo surface (lane-safe path)

```
scripts/fleet-monitor/
  README.md
  discover-urls.mjs      # registry + sitemap recursion (status-gated)
  check-critical.mjs     # T0 + T0b + cert + redirect
  check-sitemap.mjs      # T1 (polite crawl)
  report.mjs             # JSONL → daily markdown summary + heartbeat
  thresholds.json        # per-site overrides (body floor, marker, TTFB)
docs/FLEET-MONITORING-ARCHITECTURE-PROPOSAL.md   # this doc
```

Deliberately *not* `scripts/ops/` — that directory holds owner/security-lane tooling (root deploy, Caddy CA rotation, key purges). Monitoring scripts are engineering-owned, presentation-lane code and live beside `fleet-local.mjs` and `fleet-sites.json`, keeping the lane boundary clean.

Evidence output: `/var/lib/astro-fleet/monitor/` **or** repo-ignored `local/monitor-evidence/` for laptop runs — **Aaron to confirm write path on host** (recommended: `/var/lib/astro-fleet/monitor/`, owned by `webdev`, no secrets inside).

6. Layer B — Passive monitoring (crawler & real 5xx)

Synthetic checks answer: "Can *we* reach the site?"

Logs answer: "Did *Googlebot* succeed?"

6.1 Data sources (owner-controlled)

Source	Use
Cloudflare Analytics / Logpush	Edge status, bot score, cache, country, path
Caddy access/error logs	Origin status, upstream errors, timings
Optional: Node stdout `http_access`	App-level status + ms (already partially present)

6.2 Verified Googlebot (not User-Agent spoofing)

For claims like "X% of Googlebot requests failed":

1. Filter UA containing Googlebot
2. **Primary:** match source IP against Google's published ranges (`googlebot.json` / `special-crawlers.json`, refreshed daily) — cheaper and more reliable than per-request DNS
3. **Fallback:** reverse DNS → `*.googlebot.com` / `*.google.com`, forward DNS must match original IP
4. Only then count in Googlebot success/fail metrics

The same published-list approach extends to Bingbot and other verifiable crawlers later.

6.3 Metrics from logs

- 5xx rate by site / path / hour (monitor UA excluded — § 5.6)
- Googlebot verified success rate
- Spike detection vs 7-day baseline
- Correlation with `x-astro-fleet-commit` / release time when available

6.4 Owner asks (blocking for Layer B)

- ■ Cloudflare API or Logpush destination approved?
- ■ May we read `/var/log/caddy/` (or ship logs to a store)?
- ■ Retention (e.g. 14 / 30 days)?
- ■ Who receives SEO vs infra alerts?

7. Alerting & reporting

7.1 Alert classes

Severity	Example	Action
P1	Homepage or sitemap 5xx ×2 consecutive (×3 in publish window)	Page / call owner + engineering
P2	Critical latency breach sustained 15m · **missing monitor heartbeat** · cert < 7 days	Investigate CF cache / origin / runner
P3	Sitemap URL new 404s / URL-count drop / cert < 14 days	SEO + engineering review redirects/content
Digest	Daily fleet health MD/PDF incl. CrUX CWV trend + monitor uptime	Share with SEO/ops

7.2 Evidence packet (every alert)

Must include: domain, URL, status, previous status, latency, **edge-or-origin verdict** (from T0 vs T0b), fleet commit header, check id, timestamp UTC, link to evidence file.

Alert secrets: the Slack/email webhook is a credential. It lives in an owner-managed env file on the host (e.g. `/etc/astro-fleet/monitor.env`, root-owned, read by the runner) — never in the repo, matching the no-secret-sprawl rule.

7.3 Field Core Web Vitals via CrUX (free, no infra)

AARON-RULES.md already records real CWV failures (Connect HR mobile LCP ~17.7s, Connect Resources ~6.7s). The **Chrome UX Report (CrUX) API** returns Google's field CWV (LCP / INP / CLS, 75th percentile) per origin — one free HTTPS call per site per day, no Lighthouse infra, and it is the same data Google Search uses. The daily digest includes a per-site CWV table and flags regressions, closing the "speed complaints become post-mortems" gap without violating the no-fleet-wide-Lighthouse non-goal.

8. Rollout plan (phased)

Phase 0 — Approval (this doc)

Aaron confirms the decision list in § 12 (recommended defaults pre-filled — approving as-is is one word).

Phase 1 — Baseline (1 week, read-only)

- Run Layer A critical + one full sitemap pass for all **live** domains from laptop or approved host
- Publish baseline report: current 4xx/5xx/latency/cert inventory + per-site body-size floors and markers
- First CrUX pull for all origins
- No alerts yet — calibrate thresholds; seed sitemap URL-count baselines

Phase 2 — Pilot (4 sites)

Suggested pilot: `connectgroup.co`, `connecthr.ae`, `kpmpro.ae`, `peomiddleast.com`

- Enable 5-minute critical + origin checks
- Daily sitemap crawl (polite mode)
- Manual Slack/email for P1 only
- Verify publish-window muting against a real release
- Tune thresholds

Phase 3 — Fleet Layer A

- All live registry sites on T0 + T0b + T1
- Post-publish hook: after successful `publish`, queue T1 for changed domains only
- Heartbeat alerting live
- Weekly digest incl. CrUX trends

Phase 4 — Layer B

- Log ingestion + verified Googlebot dashboards
- SEO incident alerts on crawler 5xx
- Document runbooks

Phase 5 — Hardening

- Multi-region synthetic (if needed)
- Optional limited Playwright journeys
- SLOs published in ops handbook

9. Ownership matrix

Area	Engineering (presentation)	Aaron / owner
Registry-driven check scripts (`scripts/fleet-monitor/`)	Build & maintain	Review
Host cron / service for runner	Propose	Approve + install
Evidence disk path	Propose (`/var/lib/astro-fleet/monitor/`)	Approve
Cloudflare WAF allow rule for monitor UA/IP	Request	Install
Cloudflare logs / bot policy	Consume metrics	Provide access + policy
Caddy log format / shipping	Consume	Own
Alert channel + webhook secret (`/etc/astro-fleet/monitor.env`)	Draft templates	Own webhook / on-call
Publisher canary	Do not replace	Keep as source of truth at cutover
Thresholds / SEO severity	Propose	Approve P1/P2 with SEO

10. Security & lane constraints

This work must respect the autonomous release rules:

- No edits to live `/opt/Astro-production`
- No presentation-lane changes to Caddy, systemd, release tooling, or shared `packages/` without owner lane
- Monitor scripts live under `scripts/fleet-monitor/`, not `scripts/ops/` (owner/security tooling) — lane boundary stays clean
- Runner is **egress HTTPS to public sites + read-only loopback GETs** and/or **read-only log read** — not a backdoor into secrets
- No production `.env` printing; webhook secret in owner-managed env only
- Form monitors must not create uncontrolled CRM noise
- Monitor traffic is identified (UA) and excluded from analytics; crawls are rate-limited

11. Success criteria

After Phase 3:

1. Any homepage/sitemap **5xx** on a live site generates an alert within **10 minutes**, stating **edge or origin**
2. Daily report lists every sitemap URL that left **200** → **404/5xx**
3. Post-publish: changed sites get a full sitemap pass within **1 hour**, with no false page from the cutover itself
4. A dead monitor is detected within **2× its check interval** (heartbeat), never mistaken for a green fleet
5. Engineering can answer "was it 404, 500, or 502 — and at which layer?" with evidence, not guesses

After Phase 4:

6. Verified Googlebot 5xx rate is visible per site per day
7. SEO and infra share one dashboard language (status codes + rates + field CWV)

12. Decision request (Aaron)

Recommended defaults pre-filled — approve as-is or amend:

#	Decision	Recommended	Aaron
1	Architecture: Layer A (synthetic) + Layer B (logs) as above	**Yes**	____
2	Runner placement	**A — host cron under** `webdev` (enables origin probes)	____
3	Evidence path on host	`/var/lib/astro-fleet/monitor/` (webdev-owned, no secrets)	____
4	Alert destination	**Slack webhook**, secret in `/etc/astro-fleet/monitor.env` (owner-managed)	____
5	Cloudflare WAF allow rule for monitor UA + runner IP	**Yes** (one rule, owner-installed)	____
6	Cloudflare log access for Googlebot metrics	**Yes** (Phase 4)	____
7	Caddy access log read for 5xx	**Yes** (read-only, Phase 4)	____
8	PR with Phase 1 scripts under `scripts/fleet-monitor/`	**Yes**	____

13. One-line summary

Build a registry-driven, status-gated synthetic monitor (edge + origin) for all live pages/posts — identified, polite, self-monitored, publish-aware — keep the publisher canary, then add Cloudflare/Caddy log ingestion so verified-Googlebot failures and intermittent 5xx become measurable incidents, not retrospective blame.